



SmartElex Tracker Sensor

❖ Introduction –

The **Waveshare Tracker Sensor** is a **5-channel infrared line tracking sensor module** designed for robotics applications (e.g., line-following robots). It detects the presence and position of a line (usually a black line on a light surface) using infrared detectors. The module provides **analog output** from each channel corresponding to how much reflected infrared light is detected — allowing precise position calculations for tracking algorithms.

❖ Features –

- 5 infrared detectors (5-channel) for high-resolution line tracking
- High sensitivity and stable response
- Analog outputs for each channel
- Detects line presence and position using weighted readings
- Simple interface with microcontrollers like the Arduino
- Compatible with 3.3 V to 5 V systems (Arduino Uno works directly)

❖ Specifications –

Parameter	Value
Operating Voltage	3.3 V to 5 V
Detection Range	1 cm – 5 cm
Number of IR Sensors	5 channels (ITR20001/T)
Output Type	Analog outputs
Dimensions	78 mm × 18 mm
Detector Spacing	16 mm
Mounting Holes	3 mm
Weight	~7 g

❖ Pin Description –

Pin	Description
VCC	Power supply (3.3 V ~ 5 V)
GND	Ground
IR1	Analog output – Channel 1
IR2	Analog output – Channel 2
IR3	Analog output – Channel 3
IR4	Analog output – Channel 4
IR5	Analog output – Channel 5

❖ Interfacing with Arduino –

- VCC → 5V on Arduino
- GND → GND on Arduino
- IR1 → A0
- IR2 → A1
- IR3 → A2
- IR4 → A3
- IR5 → A4

The signals are **analog voltages** corresponding to IR reflection strength. You read them using `analogRead()` in Arduino.

Tip: If you only need a simple digital detection (line/no-line), you can compare analog values to a threshold in code.

- The 5 sensors are read using `analogRead()`.
- Each reading gives a value ~0–1023 (for Arduino Uno).
- Lower values typically indicate a **black line**, higher ones indicate a **white surface**.

❖ Sample Arduino Code –

```
// Arduino Uno - Line Tracking Simple Read
```

```
int sensorPins[5] = {A0, A1, A2, A3, A4};
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    for (int i = 0; i < 5; i++) {
```

```
        pinMode(sensorPins[i], INPUT);
```

```
    }
```

```
}  
  
void loop() {  
    Serial.print("Sensors: ");  
    for (int i = 0; i < 5; i++) {  
        int val = analogRead(sensorPins[i]);  
        Serial.print(val);  
        if (i < 4) Serial.print(", ");  
    }  
    Serial.println();  
    delay(200);  
}
```

❖ Calibration Tips

- Place the sensor **close (~1–5 cm)** above the surface.
- Sweep the robot back and forth over the line during startup to record min/max values per sensor.
- Use calibration data to normalize raw analog values before decision logic.

❖ Applications –

- Line FOLLOWING robots
- Maze solving robots
- Educational robotics projects
- PID control learning with sensors